FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

PROJECT REPORT

AI COLLEGE TIMETABLE & RESOURCE SCHEDULER

A PROJECT REPORT

Submitted by

DIVYA SHREE G – 240701129

JAI SHURIYA J – 240701200



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report "AI COLLEGE TIMETABLE & RESOURCE SCHEDULER" is the bonafide work of "DIVYA SHREE G (240701129) and JAI SHURIYA J (240701200)" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Dr. Mrs. Theepa Karthisai

Assistant Professor

Department of Computer Science and Engineering

Rajalakshmi Engineering College

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	1
1	INTRODUCTION	2
2	LITERATURE REVIEW	4
3	PROPOSED SYSTEM	7
4	MODULES DESCRIPTION	14
5	IMPLEMENTATION AND RESULTS	19
6	CONCLUSION AND FUTURE WORK	25
7	REFERENCES	26

TABLE OF FIGURE

FIGURE NO	TITLE	PAGE NO
Fig. 3.1	System architecture of the AI College Timetable and Resource Scheduler	7
Fig. 3.2	Logical data model	9
Fig. 3.3	Flowchart of the CSP timetable engine	11
Fig. 3.4	MRV ordering and an illustrative backtracking search tree	13
Fig. 4.1	Module overview of the system	15
Fig. 4.2	Best-fit room ranking pipeline with an example for 45 attendees	18
Fig. 4.3	Event request and approval workflow	19
Fig. 5.1	Login page of the AI Scheduler Portal	23
Fig. 5.2	Admin Control Center with timetable quality and constraint metrics	23
Fig. 5.3	Weekly teaching timetable of a faculty member	24
Fig. 5.4	Event / workshop request form with ranked room suggestion	25
Fig. 5.5	Free Room Finder showing available rooms for Period 1	26
Fig. 5.6	AI chat assistant widget on the faculty dashboard	26

TABLE OF TABLES

TABLE NUMBER	TITLE	PAGE NO
Table 2.1	Comparison of approaches	5
Table 3.1	Technology stack	8
Table 3.2	Period timings	8
Table 3.3	Hard and soft constraints	10
Table 4.1	Authentication and preference interfaces	16
Table 5.1	Software environment	21
Table 5.2	Dataset used for the experiment	22
Table 5.3	CSP configuration and run details	22
Table 5.4	Metrics reported by the Admin Control Center	23

ABSTRACT

Preparing a college timetable by hand means balancing teachers, rooms, laboratories and student batches so that no two sessions clash. The task is slow, error-prone and hard to revise. This project presents the AI College Timetable and Resource Scheduler, a full-stack system that formulates timetabling as a Constraint Satisfaction Problem (CSP) and solves it with backtracking search, domain pruning, the Minimum Remaining Values (MRV) heuristic and forward checking. The engine guarantees that no teacher, room or batch is double-booked, that laboratory sessions use laboratory rooms, that room capacity is never smaller than the batch size, and that every batch keeps a free period each day. Each placed session carries a readable explanation of the constraints behind its slot. Around the engine, role-based Student, Faculty and Admin portals provide personalised timetables, preference locking, a free-room finder, an event-request workflow that re-checks availability on approval, and workload analytics. A chat assistant built on the Google Gemini API converts questions into structured intents, answers them from live MongoDB data, and uses Gemini only to phrase the reply, so it cannot invent schedule facts. The system is implemented with FastAPI, MongoDB and React. On the seeded dataset of 6 subjects, 24 teachers and 94 rooms it scheduled 116 sessions with zero teacher, room and batch conflicts and 100% preference satisfaction.

Keywords: constraint satisfaction, backtracking, MRV, forward checking, timetabling, FastAPI, MongoDB, Gemini.

CHAPTER 1

INTRODUCTION

1.1 Background

Every semester a college must decide which class meets in which room at which time. The decision involves several kinds of resources at once: teachers who can be in only one place at a time, classrooms and laboratories of different sizes, student batches that must not have two classes together, and a fixed grid of periods in the week. A timetable is valid only when all of these resources fit together without a clash.

When this work is done manually, the coordinator typically starts from last year's timetable and patches it. One change, such as a teacher becoming unavailable, can force many other changes. Mistakes are often found only after the timetable is published. Special events and workshops make the problem harder, because they need rooms that are not already used by regular classes.

Timetabling is a classic problem in artificial intelligence. It fits the Constraint Satisfaction Problem (CSP) model, in which a solution is an assignment of values to variables that satisfies a set of constraints. The Foundations of Artificial Intelligence course introduces CSPs, backtracking search and heuristics such as Minimum Remaining Values. This project applies those ideas to a practical, working application.

1.2 Problem Statement

Given a set of student batches, each studying one subject under one teacher, a set of rooms with known capacity and type (classroom or laboratory), and a weekly grid of six periods per day, the system must produce a weekly timetable in which every session of every batch is placed in a suitable room and period without any clash. In addition, the system must let students view their own timetable, let faculty request rooms for events, let the administrator approve those requests without creating a clash, and answer everyday questions about rooms and classes in natural language.

1.3 Objectives

- To model college timetabling as a CSP and solve it automatically with backtracking, MRV and forward checking.

- To guarantee that no teacher, room or batch is double-booked, that labs and classrooms are used correctly and that room capacity is respected.
- To keep a daily break for every batch by limiting the number of sessions placed on a day.
- To explain every placement in readable language so that users can see why a slot was chosen.
- To provide separate Student, Faculty and Admin portals with secure, role-based access.
- To allocate rooms by best fit and to handle event requests with a real-time availability check.
- To provide a chat assistant whose answers always come from real database records.

1.4 Scope and Contributions

The system covers one college campus with 6 subjects, 24 teachers, 94 rooms and six periods per day. The main contributions of this work are:

- A custom CSP engine with domain pruning, MRV ordering, forward checking, soft-constraint value ordering and an independent verification step.
- A single schedule collection shared by timetable sessions and approved events, so that room availability is always consistent.
- A grounded chat pipeline in which Gemini translates language into structured intent and phrases the answer, while the database supplies every fact.
- An administrator dashboard that reports conflicts, preference satisfaction, workload and utilisation directly from stored data.

The rest of this report is organised as follows. Chapter 2 reviews related work. Chapter 3 describes the proposed system, its architecture, database and algorithm. Chapter 4 explains each module. Chapter 5 presents the experimental setup and results with output screens. Chapter 6 concludes the report and lists future work.

CHAPTER 2

LITERATURE REVIEW

This chapter surveys work on timetabling, constraint satisfaction, metaheuristic methods and grounded language assistants, which are the four areas this project draws on.

2.1 Timetabling as a Computational Problem

Even, Itai and Shamir (1976) proved that timetable design is NP-complete, which means that no known algorithm solves all instances quickly. Practical systems therefore rely on heuristics that exploit the structure of real data.

Schaerf (1999) surveyed automated timetabling and separated it into school, university course and examination timetabling, each with different constraints. Burke and Petrovic (2002) discussed recent research directions and stressed the difference between hard constraints, which must never be violated, and soft constraints, which are preferences to be satisfied as far as possible. The same distinction is used in this project.

2.2 Constraint Satisfaction Approaches

Russell and Norvig (2020) define a CSP by a set of variables, a domain of values for each variable and a set of constraints. They describe backtracking search and two key improvements: the Minimum Remaining Values heuristic, which picks the variable with the fewest legal values, and forward checking, which removes inconsistent values from the domains of unassigned variables after each assignment.

The theoretical basis for these ideas comes from earlier work. Mackworth (1977) introduced consistency algorithms for networks of relations, which led to constraint propagation. Haralick and Elliott (1980) showed that forward checking combined with a fail-first ordering greatly reduces tree search effort. Tsang (1993) and Dechter (2003) give book-length treatments of constraint satisfaction and constraint processing.

In timetabling, Abdennadher and Marte (2000) applied constraint handling rules to university course timetabling and showed that a declarative constraint formulation can express real timetabling rules clearly. This supports the choice made here to express room, teacher and batch rules as constraints rather than as ad hoc code.

2.3 Metaheuristic and Hybrid Approaches

Lewis (2008) reviewed metaheuristic techniques for university timetabling, including genetic algorithms, simulated annealing and tabu search. These methods improve a complete timetable step by step and handle soft constraints well, but they do not by themselves guarantee that every hard constraint is satisfied, and they cannot prove that no solution exists.

For this project the hard constraints are the most important requirement, because a clash means that a class cannot actually be held. A complete search such as backtracking either returns a timetable that satisfies all hard constraints or reports that none exists. Soft constraints are handled by trying preferred values first. This keeps the system simple while giving the guarantee that matters most.

2.4 Grounded Language Assistants

Large language models can answer questions fluently but may state facts that are not true. Lewis et al. (2020) proposed retrieval-augmented generation, in which a model is given retrieved information and uses it when producing an answer. The Gemini family of models (Gemini Team, 2023) supports instruction following and structured output, which makes it suitable for a controlled pipeline.

The chat assistant in this project follows the same principle in a simpler form. Instead of retrieving text passages, the backend runs an exact database query chosen from the intent that Gemini extracts, and Gemini is then asked to phrase only the returned data.

2.5 Summary and Research Gap

Table 2.1 Comparison of approaches

Approach	Strength	Limitation	Use in this project
Manual or spreadsheet planning	No special tools needed	Slow, clashes found late, hard to change	Replaced by automation
CSP with backtracking, MRV and forward checking	Hard constraints exact; reports failure honestly	Worst case exponential on very crowded data	Core timetable engine
Metaheuristics (GA, SA, tabu search)	Good at optimising soft preferences	No guarantee of hard-constraint satisfaction	Not used; listed as future work
Language model used alone	Natural, fluent answers	May invent facts	Used only to parse and phrase

Most published work concentrates on the solver alone. Few descriptions combine a conflict-free solver with the daily needs of a college: personal timetables for students, room requests from faculty, approval workflows for administrators and a natural-language interface that cannot mislead. This project addresses that gap by integrating all of these around one shared schedule.

CHAPTER 3

PROPOSED SYSTEM

The proposed system is a three-tier web application. A React front end talks to a FastAPI back end over a REST interface secured with JSON Web Tokens. The back end calls three intelligence components, namely the CSP engine, the room-allocation service and the Gemini client, and stores all data in MongoDB.

3.1 Architecture Diagram

Figure 3.1 shows the overall architecture. Users of three kinds (student, faculty and administrator) work through one React application. After login, the application shows pages and menus that match the user's role.

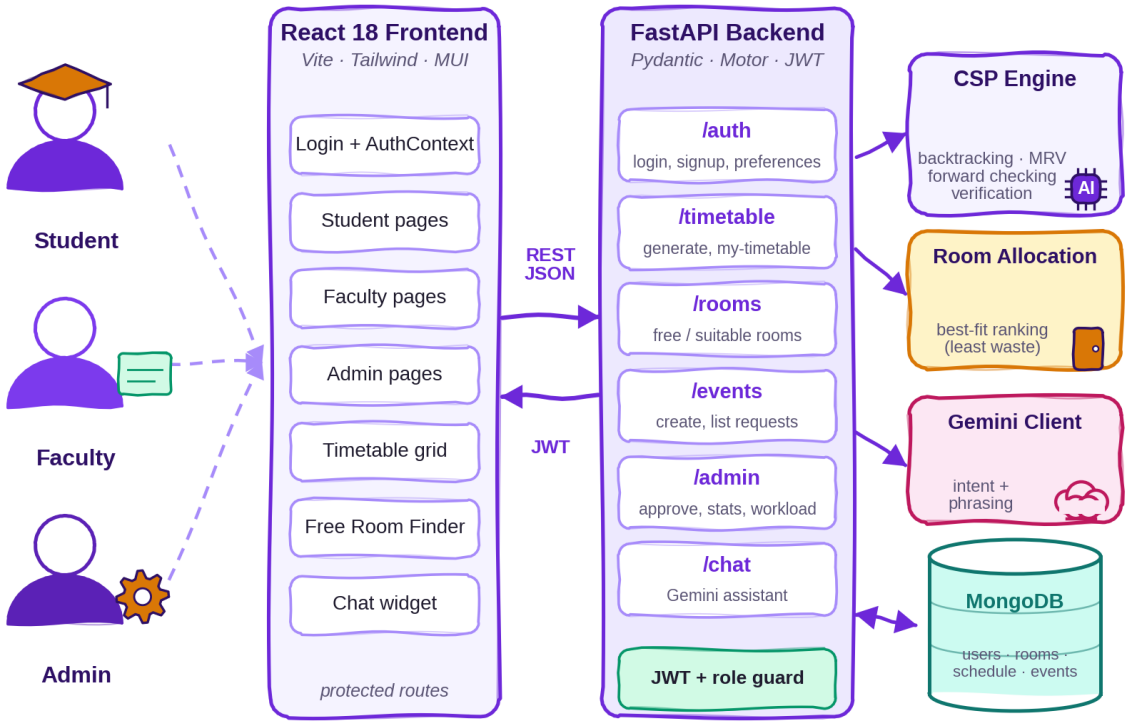


Fig. 3.1 System architecture of the AI College Timetable and Resource Scheduler

The architecture has four layers:

- **Presentation layer.** React 18 with Vite, Tailwind CSS and Material UI. React Router protects each page by role, and an authentication context keeps the logged-in user and token.

- **Application layer.** FastAPI exposes six groups of routes: auth, timetable, rooms, events, admin and chat. Pydantic models validate every request.
- **Intelligence layer.** The CSP engine builds the timetable, the room-allocation service ranks rooms, and the Gemini client turns language into intent and data into sentences.
- **Data layer.** MongoDB, accessed through the asynchronous Motor driver, stores users, subjects, teachers, batches, rooms, schedule entries and event requests.

Table 3.1 Technology stack

Layer	Technology	Purpose
Front end	React 18, Vite, Tailwind CSS, MUI, React Router	User interface, routing, role-based pages
Back end	Python, FastAPI, Pydantic	REST API and request validation
Security	JWT, bcrypt	Authentication, authorisation, password hashing
Database	MongoDB with Motor	Persistent storage of all records
AI engine	Custom CSP solver	Conflict-free timetable generation
Language model	Google Gemini API	Intent extraction and answer phrasing

3.2 Database Design

Figure 3.2 shows the logical collections and the main fields of each. The field names are given as key attributes and may differ slightly from the exact names in the code.

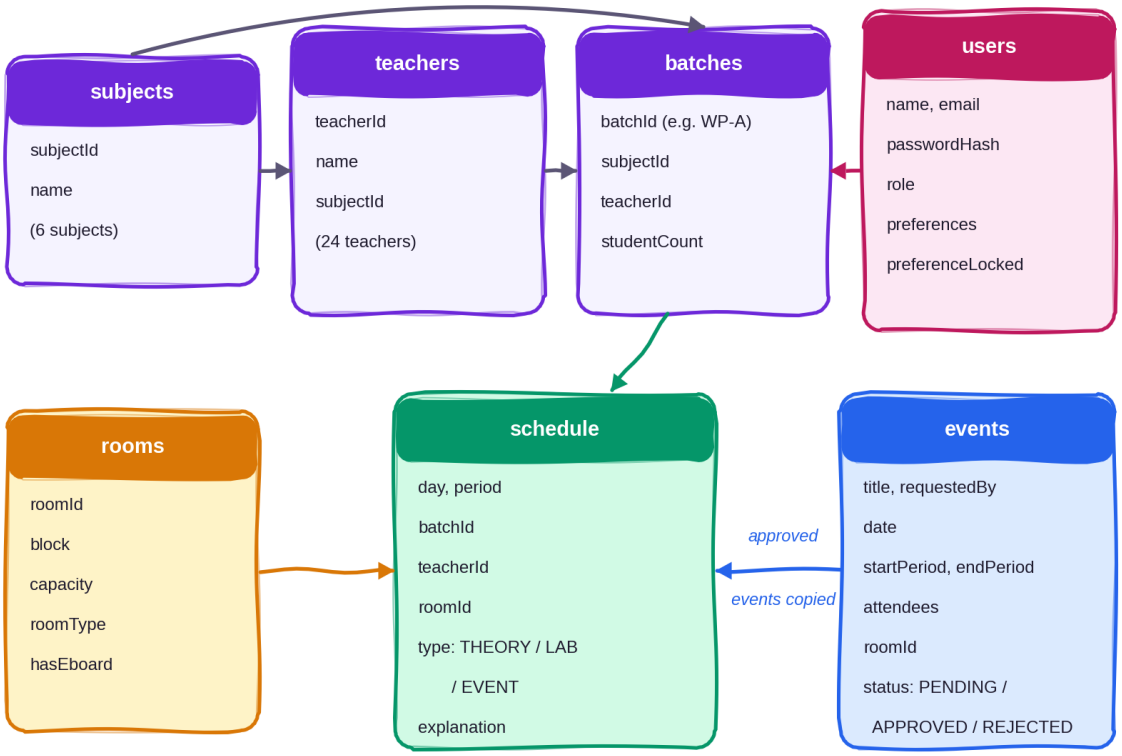


Fig. 3.2 Logical data model (key fields of each collection)

Subjects, teachers and batches describe what must be taught and by whom. Each batch belongs to one subject, has exactly one teacher and has a student count. Rooms store block, capacity, type and whether a smart e-board is present. Users store credentials, role, chosen preferences and a lock flag.

The most important design decision is the single schedule collection. Regular class sessions and approved events are both stored there. A room is therefore free only if the schedule has no entry for it at that day and period, and the same check serves the timetable engine, the free-room finder and the event approval step.

3.3 CSP Formulation

Variables

A variable is one session that a batch needs in a week. For example, a batch may need three theory sessions of one period each and two lab sessions of two consecutive periods each. A lab is treated as one variable of length two, so both periods are placed together in the same room. The teacher is not searched for, because each batch already has exactly one teacher, which is attached to every variable of that batch.

Domains

The domain of a variable is the set of triples (day, period, room). Before the search, static rules remove triples that can never work. The time grid has six periods per day, as shown in Table 3.2, and classes run from Monday to Saturday.

Table 3.2 Period timings

Period	Time	Period	Time
P1	09:00 – 09:50	P4	12:00 – 12:50
P2	10:00 – 10:50	P5	14:00 – 14:50
P3	11:00 – 11:50	P6	15:00 – 15:50

Constraints

Table 3.3 Hard and soft constraints

ID	Constraint	Type
C1	A teacher has at most one session at a given day and period	Hard
C2	A room has at most one session at a given day and period	Hard
C3	A batch has at most one session at a given day and period	Hard
C4	Room capacity is at least the batch student count	Hard
C5	Lab sessions use lab rooms; theory sessions use classrooms	Hard
C6	A two-period lab cannot start at the last period of a day, and both of its periods must pass C1 to C3	Hard
C7	Every batch has at most five sessions on a day, so at least one period is free	Hard
C8	Preferred teacher, preferred room block, morning slots	Soft

The daily-break rule C7 is applied per batch and not for the whole college. One batch may be free in period 3 while another is free in period 5, so no single lunch hour is forced and room demand is spread out over the day.

3.4 CSP Algorithm

The engine works in the following steps, which are also summarised in Figure 3.3.

1. **Build variables.** Create one variable per session of every batch.
2. **Prune domains.** Remove (day, period, room) values that violate C4, C5 or C6 before any search.
3. **Select a variable.** Use the MRV heuristic: choose the unassigned variable with the fewest legal values.
4. **Try values.** Take values in an order that puts soft-constraint preferences first. Accept a value only if C1, C2, C3 and C7 hold with the current assignment.
5. **Forward check.** After assigning, remove the chosen slot and room from related domains. If any domain becomes empty, undo the assignment immediately.
6. **Backtrack.** If no value works, return to the previous variable and try its next value.
7. **Verify and enrich.** When all variables are assigned, re-scan the timetable for conflicts and add times, duration and an explanation for each session.

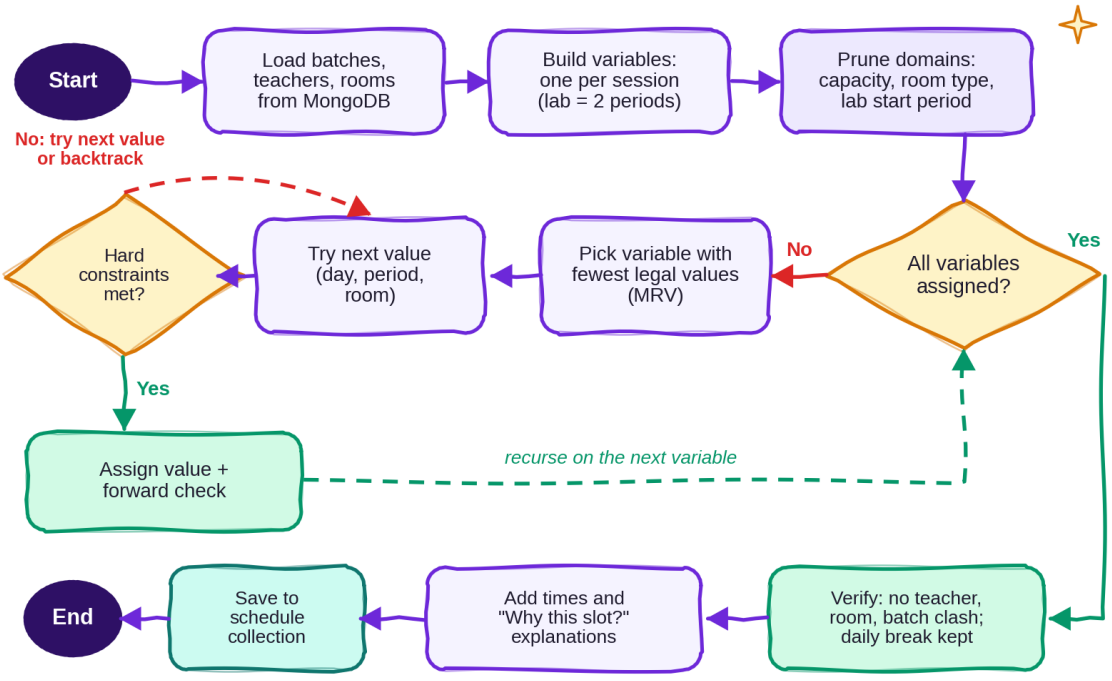


Fig. 3.3 Flowchart of the CSP timetable engine

Algorithm 3.1 Backtracking search with MRV and forward checking

```
function BACKTRACK(assignment):
    if every variable is assigned:
        return assignment
    var <- MRV(unassigned variables)
    for value in ORDER_VALUES(var):        # soft
preferences first
        if CONSISTENT(var, value, assignment):
            assignment[var] <- value
            if FORWARD_CHECK(var, value):  # no empty
domain
                result <- BACKTRACK(assignment)
                if result is not failure:
                    return result
            UNDO_FORWARD_CHECK(var, value)
            delete assignment[var]
    return failure
```

Why MRV helps

A variable with only a few legal values is the most likely to fail. Placing it first exposes a dead end early, when little work has been done. In practice, lab sessions, which can use only lab rooms, are placed first and flexible theory sessions are placed around them. This is the same idea as starting a Sudoku puzzle with the most constrained cell. Figure 3.4 illustrates the ordering and a short backtracking example.

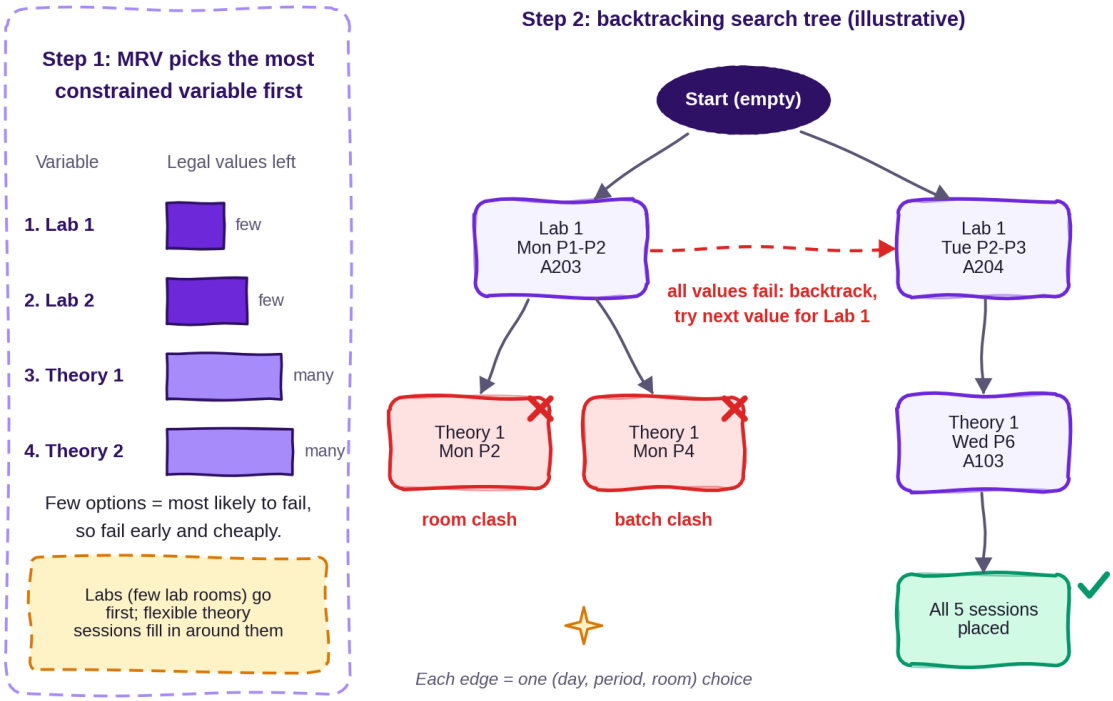


Fig. 3.4 MRV ordering and an illustrative backtracking search tree

Effect of domain pruning

Pruning shrinks the search space before the search starts. For example, a lab session for a 55-student batch begins with every combination of slot and room. After the static rules, only lab rooms with at least 55 seats remain, which is a small fraction of the original options.

Complexity

Timetabling is NP-hard, so the worst case is exponential. Domain pruning, MRV and forward checking make the search practical on the data used here. If a dataset is too crowded to have any valid solution, the algorithm exhausts all options and reports failure; it never produces an invalid timetable to hide the problem.

3.5 Verification and Explainability

After a solution is found, a separate function, `verify_no_conflicts()`, scans the finished timetable and checks that no teacher is in two places at once, no room is double-booked, no batch has overlapping sessions and every batch has at least one empty period on each day. Because this check is independent of the search code, it can catch bugs in the solver itself. The “Conflicts” figure on the administrator dashboard comes from the same check and should always read zero.

Every scheduled session also stores a short human-readable explanation. The “Why this slot?” button on each timetable card shows the constraints that led to the placement, such as the room type, capacity and the free time of the teacher and batch.

CHAPTER 4

MODULES DESCRIPTION

The system is divided into eight modules. Figure 4.1 shows how the three role-based portals sit on top of a shared REST layer and four core services.

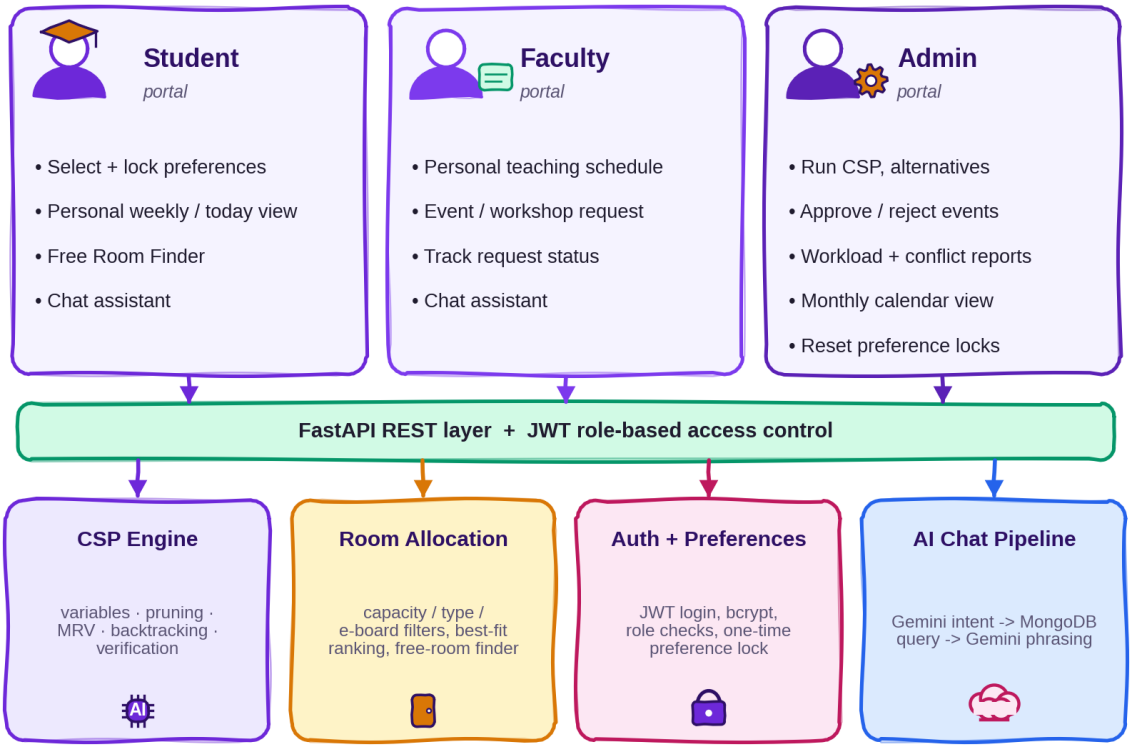


Fig. 4.1 Module overview of the system

4.1 Module 1: CSP Timetable Engine

This is the core module. It reads batches, teachers and rooms from MongoDB, builds the session variables, prunes their domains and runs the backtracking search described in Chapter 3. The result is written to the schedule collection.

- **Input:** batches with subject, teacher and student count; rooms with capacity and type; the period grid.
- **Processing:** domain pruning, MRV selection, forward checking, backtracking and soft-constraint value ordering.
- **Output:** a conflict-free weekly schedule with start time, end time, duration and an explanation for each session.

- **Alternatives:** the administrator can generate an alternative timetable, which runs the same engine with a different random seed so that value ordering changes.
- **Interfaces:** `POST /timetable/generate`, `POST /admin/timetable/alternative` and `GET /admin/conflicts`.

4.2 Module 2: Authentication and Preferences

Users sign up or log in with an email and password. Passwords are hashed with bcrypt and never stored in plain text. A successful login returns a JSON Web Token that the front end attaches to every request, and the back end checks the role in the token before allowing access to a route.

Student preferences work as follows. Each subject has more than one teacher. A student chooses the preferred teacher for each subject and, where that teacher has two batches, the section. On submission the preference lock is set, and further submissions are rejected until an administrator resets the lock. The timetable is not generated for each student. It is generated once for all batches, and each student's view is a filter over the result.

Table 4.1 Authentication and preference interfaces

Method	Endpoint	Purpose
POST	/auth/signup	Register a new user
POST	/auth/login	Log in and receive a JWT
GET	/auth/me	Current user information
POST	/preferences/submit	Save and lock student preferences
GET	/preferences/my	View own preferences
POST	/admin/users/{id}/reset-preference	Admin resets a preference lock

4.3 Module 3: Student Portal

- Select and lock preferred faculty and section for every subject.
- View a personalised weekly timetable and a today view, available after the preferences are locked.

- Use the Free Room Finder to search for unoccupied rooms by date, period and block.
- Ask the chat assistant about rooms and classes.

4.4 Module 4: Faculty Portal

- View a personal teaching timetable showing subject, room, time and class type for every session.
- Open “Why this slot?” on any session to read the constraints behind it.
- Submit event or workshop room requests (Module 7).
- Track the status of submitted requests as pending, approved or rejected.

4.5 Module 5: Admin Portal

The administrator uses the Admin Control Center to run and monitor the system.

- Run the CSP engine or generate an alternative timetable with one click.
- View quality metrics: teacher, room and batch conflicts, lab assignments, unassigned sessions and preference satisfaction.
- View teacher workload analytics: theory hours, lab hours, total weekly hours and overload detection.
- Approve or reject event requests and view a monthly calendar density map.
- Manage student preference locks by resetting them when required.

4.6 Module 6: Room Allocation

One shared function ranks rooms for the Free Room Finder and for event suggestions. It filters rooms by capacity, room type and e-board requirement, and then sorts them by the number of wasted seats, defined as room capacity minus the required capacity. A request for 45 people therefore prefers a 52-seat room over a 200-seat hall, which leaves large halls free for large events. Figure 4.2 shows the pipeline with an example.

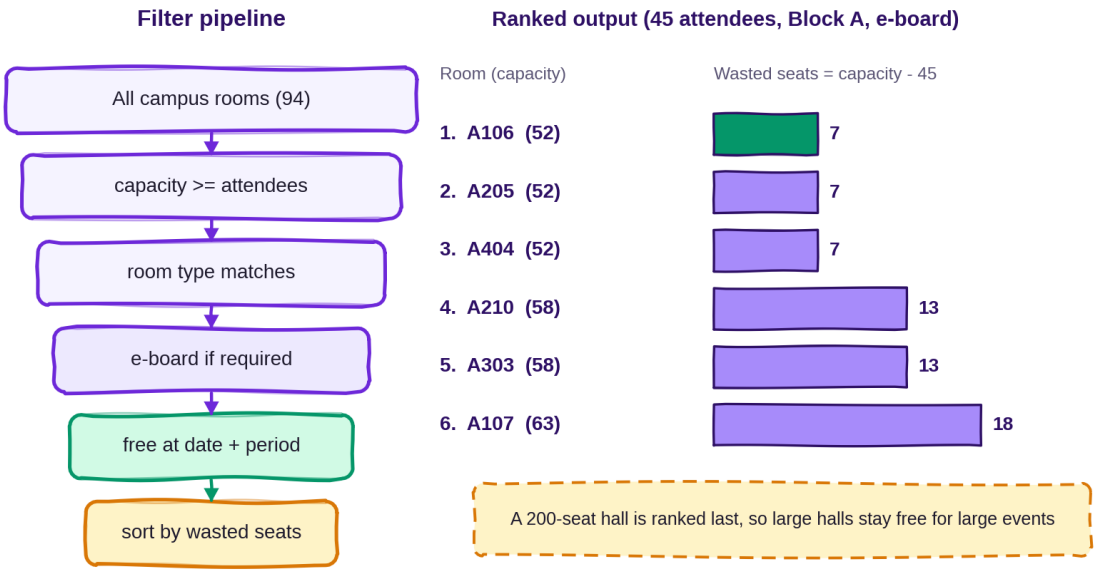


Fig. 4.2 Best-fit room ranking pipeline with an example for 45 attendees

The Free Room Finder adds one more step before ranking. It looks up which rooms are occupied at the requested day and period in the schedule collection, which holds both classes and approved events, and removes them. The related interfaces are `GET /rooms/free`, `GET /rooms/suitable`, `GET /rooms/all` and `GET /rooms/{room_id}`.

4.7 Module 7: Event Request and Approval

1. This module uses rule-based validation. The teacher enters the date, start and end period, number of attendees, preferred block, lab need and e-board need.
2. The front end blocks weekend dates immediately. The back end checks the day of the week again and returns an error for Saturday or Sunday, so the rule cannot be bypassed.
3. The system returns the rooms that are free at that date and time and match the capacity, type and e-board need, ranked by best fit. The teacher selects one room.
4. The request is stored with status PENDING and the chosen room.
5. On approval, the admin portal checks again that the room is still free, because it may have been booked since the request was made. If it is free, an EVENT entry is written to the schedule and the status becomes APPROVED. Otherwise an error is shown and the request can be rejected with a reason.

Figure 4.3 shows the workflow.

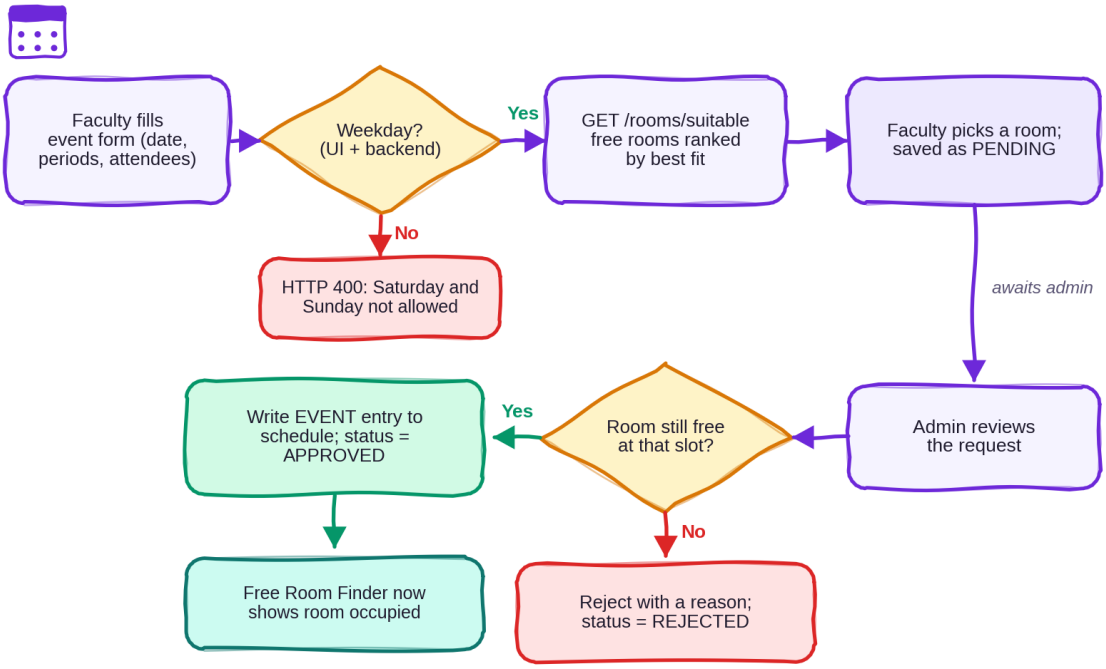


Fig. 4.3 Event request and approval workflow

Because classes and events share one schedule collection, an approved event immediately makes the room appear as occupied in the Free Room Finder.

4.8 Module 8: AI Chat Assistant

The chat assistant is available as a floating widget on every page. It answers questions about today's timetable, free rooms, room details and the schedule of a class, subject or batch. Figure 4.4 shows the pipeline.

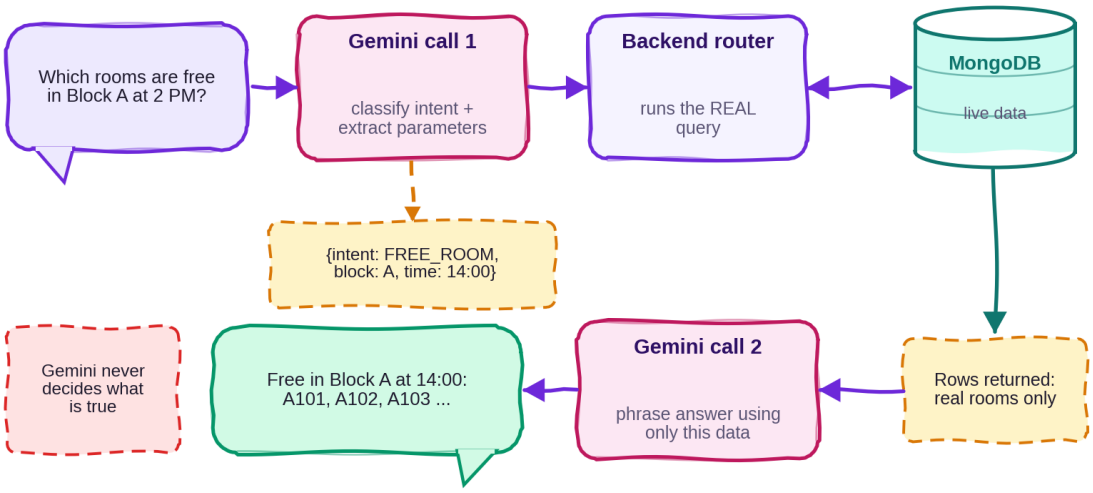


Fig. 4.4 Grounded chat pipeline (example answer is illustrative)

1. **Gemini call 1** classifies the intent and extracts parameters, such as block and time, and returns only JSON. Example prompts in the instruction keep the output consistent, and any parsing failure falls back to an UNKNOWN intent.
2. **The backend router** runs the matching real query on MongoDB.
3. **Gemini call 2** is told to answer using only the data it receives and to say so if the data is empty.

The safety of the design comes from this split of responsibilities. Gemini never decides what is true about the schedule, so it cannot invent a room or a class.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

The system was implemented and tested on a development machine. The back end runs with Uvicorn on port 8000 and the React front end runs on the Vite development server on port 5173. Table 5.1 lists the software environment.

Table 5.1 Software environment

Component	Version / tool
Python	3.9 or later
Back end framework	FastAPI with Uvicorn
Database	MongoDB (local or Atlas) with Motor driver
Node.js	18 or later
Front end	React 18, Vite, Tailwind CSS, Material UI, React Router
Security	JWT authentication, bcrypt
Language model	Google Gemini API

The database is filled by a seeding script. Table 5.2 shows the dataset that was used for all results in this chapter.

Table 5.2 Dataset used for the experiment

Item	Value
Subjects	6 (Low Code, Theory of Computation, Mobile Computing, Web Programming, Computer Networks, Foundation of AI)
Teachers	24
Batches	24, with non-uniform student counts
Rooms	94 across four blocks (A, B, Techlounge, Ideafactory)
Event requests	3 pending, 1 pre-approved
Demo users	5 (1 admin, 2 students, 2 faculty)

Table 5.3 CSP configuration and run details

Parameter	Value
Periods per day	6 (P1 to P6)
Working days for classes	Monday to Saturday
Theory session length	1 period (50 minutes)
Lab session length	2 consecutive periods (shown as 1 h 40 min)
Number of variables (sessions)	116
CSP solve time	_____ seconds (to be filled after measurement)

The experiment follows four steps: seed the database, generate the timetable with the CSP engine, start the back end and front end, and log in with each role to inspect the outputs and the quality metrics.

5.2 Results

5.2.1 Login Page

Figure 5.1 shows the login page. Users sign in with their college email and password, and the demo buttons fill in the credentials for the Student, Faculty and Admin roles for quick testing. After login, the application opens the portal that matches the role.

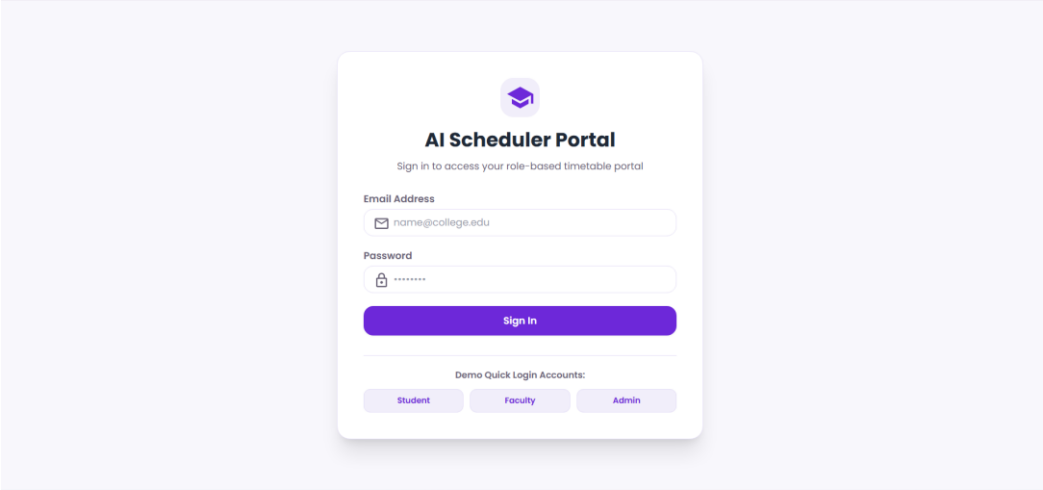


Fig. 5.1 Login page of the AI Scheduler Portal

5.2.2 Admin Control Center

Figure 5.2 shows the Admin Control Center. The buttons at the top run the CSP engine and generate an alternative timetable. The cards summarise the system, and the quality panel is calculated directly from the schedule entries stored in MongoDB.

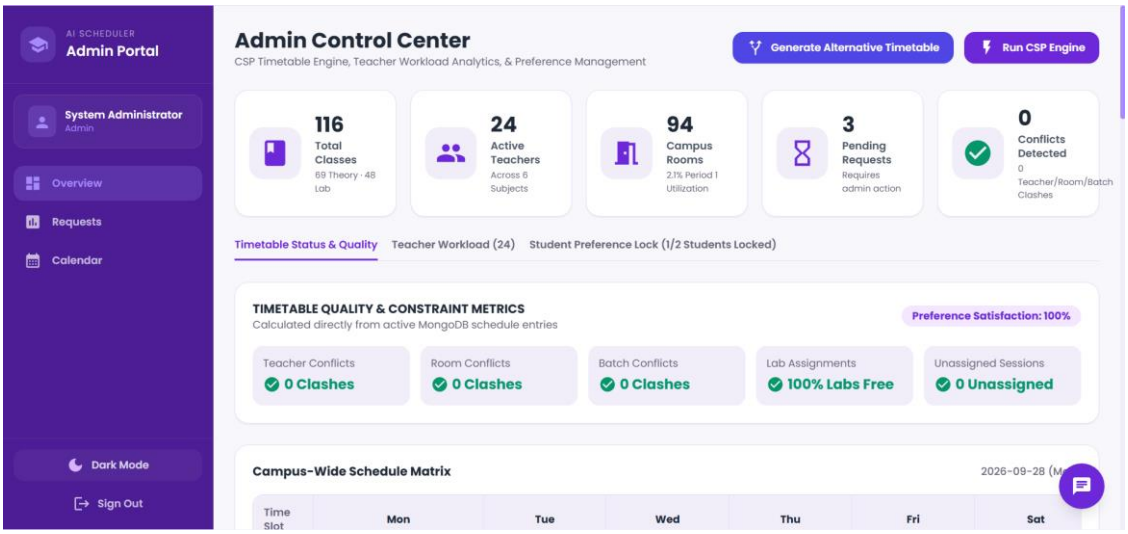


Fig. 5.2 Admin Control Center with timetable quality and constraint metrics

Table 5.4 Metrics reported by the Admin Control Center

Metric	Observed value
Total classes scheduled	116
Active teachers	24 across 6 subjects
Campus rooms	94
Teacher conflicts	0 clashes
Room conflicts	0 clashes
Batch conflicts	0 clashes
Unassigned sessions	0
Lab assignments	100% (dashboard label “100% Labs Free”)
Preference satisfaction	100%
Pending event requests	3
Period 1 room utilisation	2.1%
Student preference locks	1 of 2 students locked

5.2.3 Faculty Timetable

Figure 5.3 shows the weekly timetable of a faculty member. Each card shows the subject, teacher, room, time and class type. Theory sessions occupy one period of 50 minutes, and lab sessions occupy two consecutive periods and are shown as 1 h 40 min. For example, the Low Code lab on Friday uses period P2 in room A203 from 10:00 to 11:50. No two cards share a period, which confirms that the teacher is never double-booked. The “Why this slot?” button on each card shows the explanation for the placement.

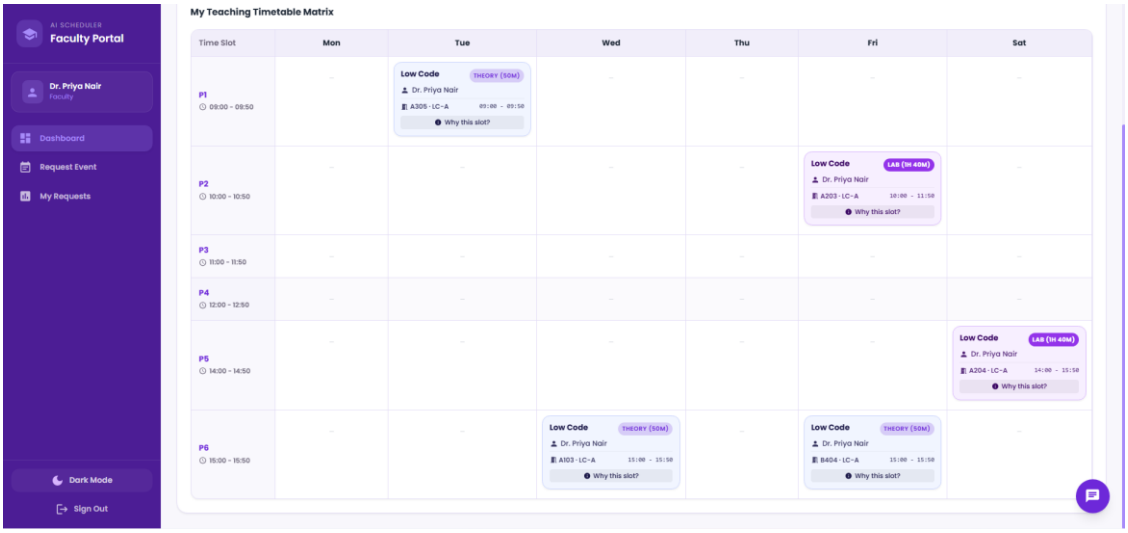


Fig. 5.3 Weekly teaching timetable of a faculty member

5.2.4 Event Request Form

Figure 5.4 shows the event request form. The faculty member enters the title, date, start and end period, expected attendees, preferred block and the lab and e-board needs. The date field is limited to Monday to Friday. The room list is filtered and ranked by capacity fit, so for 45 attendees the 52-seat rooms appear before the 58-seat and 63-seat rooms.

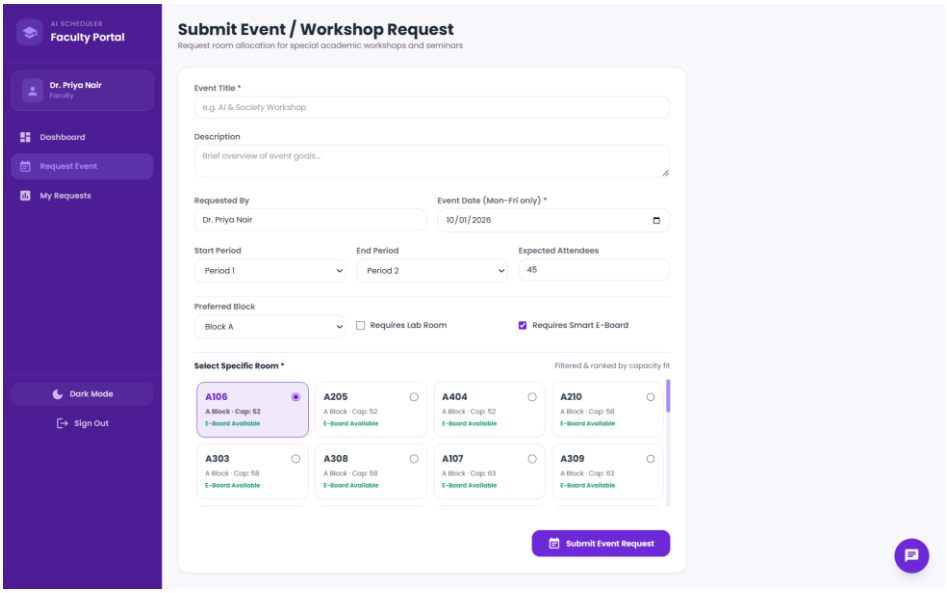


Fig. 5.4 Event / workshop request form with ranked room suggestions

5.2.5 Free Room Finder

Figure 5.5 shows the Free Room Finder in the student portal. For 28 September 2026 and Period 1, it found 92 available rooms out of 94 campus rooms. Each card shows the room name, type, block, capacity and e-board availability. The two remaining rooms were

occupied by classes in that period, which matches the 2.1% Period 1 utilisation reported on the administrator dashboard.

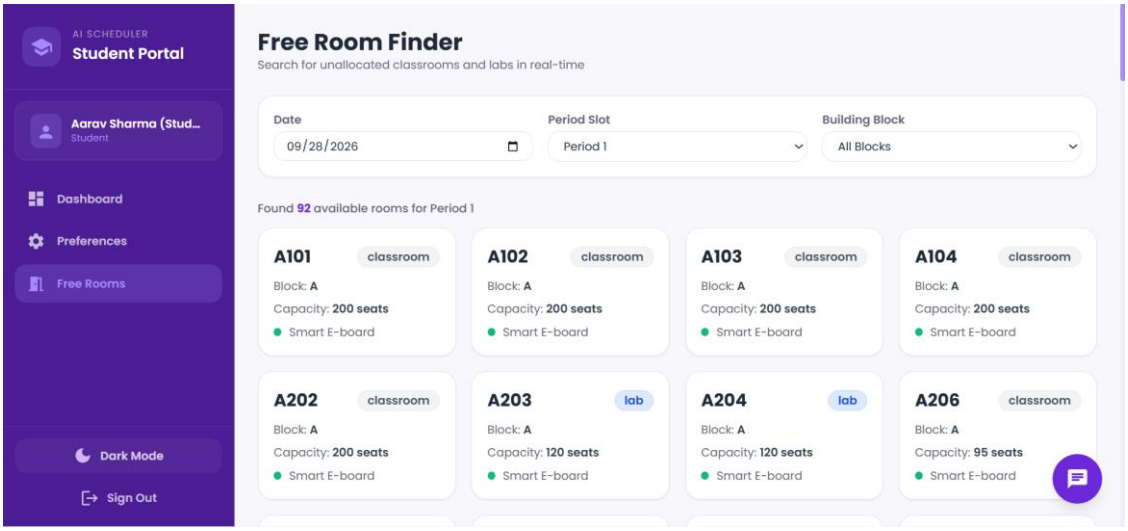


Fig. 5.5 Free Room Finder showing available rooms for Period 1

5.2.6 AI Chat Assistant

Figure 5.6 shows the chat assistant opened on the faculty dashboard. It is labelled as powered by Gemini and greets the user with the kinds of questions it can answer: free rooms, schedule details and today's classes. Questions are processed with the two-call pipeline of Module 8, so every answer is based on records returned from MongoDB.

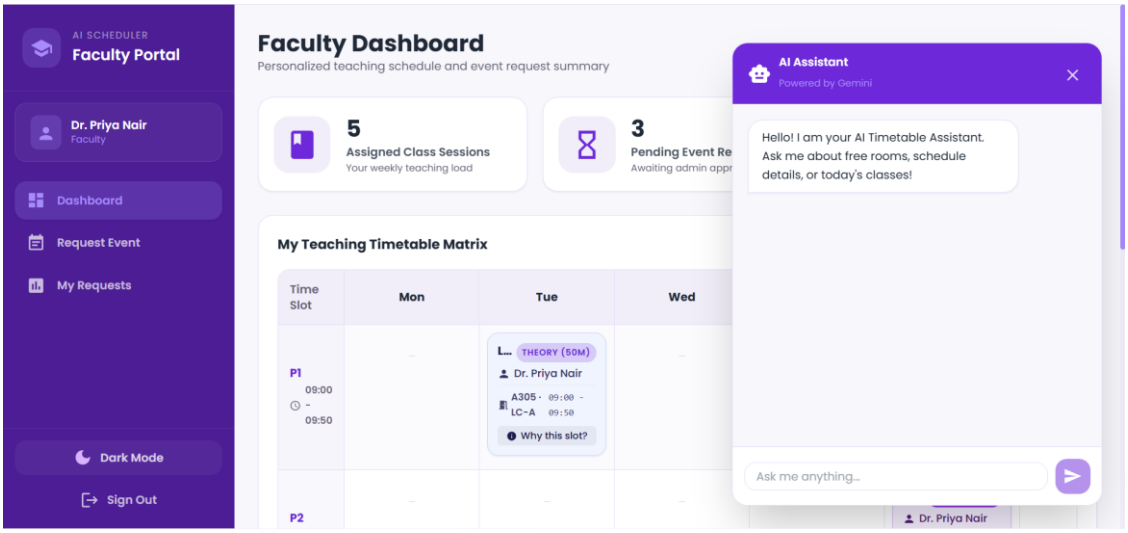


Fig. 5.6 AI chat assistant widget on the faculty dashboard

5.2.7 Discussion

The results show that the engine produced a complete timetable for 116 sessions with no teacher, room or batch conflicts and no unassigned sessions. Hard constraints are

guaranteed by the search and confirmed by the independent verification step, and the dashboard figures are computed from stored data rather than typed in. Preference satisfaction of 100% shows that soft preferences could also be met on this dataset.

The results have limits. They were obtained on a single seeded dataset, and the solve time has not yet been recorded, so the report does not claim a speed figure. A larger or more crowded dataset would be needed to study how the search time grows.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project built a complete scheduling system around a classical AI search method. Timetabling was modelled as a Constraint Satisfaction Problem and solved with domain pruning, backtracking, the MRV heuristic and forward checking. On the seeded dataset the engine scheduled 116 sessions for 24 teachers and 94 rooms without any teacher, room or batch conflict, with every lab session in a lab room and with each batch keeping a free period each day.

Around the engine, the system provides role-based portals, a one-time preference lock, best-fit room ranking, an event request workflow that re-checks availability before approval, and a dashboard that reports quality metrics from live data. The chat assistant uses Gemini only to understand questions and to phrase answers, while MongoDB supplies every fact, which removes the risk of invented schedule information.

The work shows that a small, well-chosen classical algorithm, combined with careful software design, can solve a real administrative problem and remain explainable to its users.

6.2 Future Work

- Add weighted soft constraints and a local-search repair step to improve preference quality after the first solution.
- Support incremental re-solving so that a change in one teacher or room does not require regenerating the whole timetable.
- Allow several teachers per batch, elective groups and combined classes.
- Send email or push notifications when event requests are approved or rejected.
- Export timetables to PDF and calendar (iCal) formats.
- Benchmark the solver on larger datasets and record solve time and success rate.

CHAPTER 7

REFERENCES

1. Abdennadher, S. and Marte, M. (2000) 'University Course Timetabling Using Constraint Handling Rules', Applied Artificial Intelligence, Vol.14, No.4, pp.311-325.
2. Burke, E.K. and Petrovic, S. (2002) 'Recent Research Directions in Automated Timetabling', European Journal of Operational Research, Vol.140, No.2, pp.266-280.
3. Dechter, R. (2003) Constraint Processing, Morgan Kaufmann, San Francisco.
4. Even, S., Itai, A. and Shamir, A. (1976) 'On the Complexity of Timetable and Multicommodity Flow Problems', SIAM Journal on Computing, Vol.5, No.4, pp.691-703.
5. Gemini Team, Google (2023) 'Gemini: A Family of Highly Capable Multimodal Models', arXiv preprint arXiv:2312.11805.
6. Haralick, R.M. and Elliott, G.L. (1980) 'Increasing Tree Search Efficiency for Constraint Satisfaction Problems', Artificial Intelligence, Vol.14, No.3, pp.263-313.
7. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020) 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', Advances in Neural Information Processing Systems, Vol.33, pp.9459-9474.
8. Lewis, R. (2008) 'A Survey of Metaheuristic-Based Techniques for University Timetabling Problems', OR Spectrum, Vol.30, No.1, pp.167-190.
9. Mackworth, A.K. (1977) 'Consistency in Networks of Relations', Artificial Intelligence, Vol.8, No.1, pp.99-118.
10. Russell, S. and Norvig, P. (2020) Artificial Intelligence: A Modern Approach, 4th edition, Pearson, Hoboken.
11. Schaerf, A. (1999) 'A Survey of Automated Timetabling', Artificial Intelligence Review, Vol.13, No.2, pp.87-127.
12. Tsang, E. (1993) Foundations of Constraint Satisfaction, Academic Press, London.